

Project Report

Student Details

Name: Aman Pandey
Roll No. : 23f3002644
Email-id : 23f3002644@ds.study.iitm.ac.in
Course : MAD- 1 Project
Project : Vehicle Parking App

Greetings everyone I am Aman Pandey, from New Delhi. Right now I am 21 years old and pursuing BS in Data Science and Programming from IIT Madras, along with B.Sc.(H) Mathematics from Atma Ram Sanatan Dharma College, University of Delhi.

Description

A multi-user web application for managing 4-wheeler parking lots, built with Flask, Jinja2, HTML, CSS, Bootstrap and SQLite for databases. It has two distinct user roles: **Admin** (superuser) and **Regular User**. The application provides functionalities for admins to manage parking infrastructure and for users to reserve and vacate parking spots efficiently.

% Uses of A.I.(Artificial Intelligence) :-

Category	A.I. %	Self %
Backend (Flask + DB + models +Controllers)	20%	80%
Frontend (HTML,CSS, Bootstrap,Jinja)	30%	70%

Auth + CRUD Logic	10%	90%
Extras (Charts, Search)	50%	50%

Technologies Used

The project adheres to the specified mandatory frameworks to ensure consistency and ease of deployment.

Backend

- **Flask:** A lightweight Python web framework used for building the application's backend logic, handling routing, and managing data flow.

Frontend

- **Jinja2:** A modern and designer-friendly templating language for Python, used to generate dynamic HTML content.
- **HTML:** The standard markup language for creating web pages.
- **CSS:** A stylesheet language used for describing the presentation of a document written in HTML.
- **Bootstrap:** A popular front-end framework for developing responsive and mobile-first websites, used to enhance the application's user interface and provide a consistent design.

Database

- **SQLite:** A self-contained, high-reliability, embedded, full-featured relational database management system. It's chosen for its simplicity and ease of integration for local machine demonstrations. The database will be created programmatically to ensure consistency and avoid manual intervention.

DB Schema Design

The application uses **SQLite** with the following models:

- Users (id, email, password, fullname, address, pincode, type [default= user])
- Lots (id, location, price, available_spots, total_spots, address, pincode, details [Relationship to Spots])
- Spots (id, lot_id [Foreign key{lots.id}] , status [default "Available", can be "Unavailable])
- Reserve (id, user_id, lot_id, spot_id, start_time, end_time, total_time, location, status, cost, vehicle_no)

Explaining Above:

1. "Users" Table records the details of the users, having default= user. It contains the only one admin which has type = 'admin'.
2. "Lots" Table records the details of the Lots, it also has the relationship with "Spots" Table by 'details' key.
3. "Spots" Table records the details of the spots.
4. "Reserve" Table records the details of the reservation/Booking made by the user for vehicle parking.

Architecture and Features

The Project folder Structure:

```

Vehicle_Parking_app
|
|----app.py
|----application
|      |--controllers.py
|      |--database.py
|      |--models.py
|
|----static
|      |--bar.png
|      |--css
|          |--style.css
|
|----instance
|      |--parking.sqlite3
|
|----templates
|      |--add_lot.html
|      |--admin_dash.html

```

```
|      |--admin_user.html
|      |--all_reserve.html
|      |--Booking.html
|      |--delete_spot.html
|      |--edit_lot.html
|      |--edit_user_prof.html
|      |--invalid_login.html
|      |--login.html
|      |--parking_spot_details.html
|      |--register.html
|      |--release.html
|      |--search.html
|      |--summ_admin.html
|      |--summ_user.html
|      |--user_dash.html
|
|----README.md
|----requirement.txt
```

Notes:

- Kept all python code in the application package.
- All HTML goes in templates.
- All CSS / images go in static.
- The database file (parking.sqlite3) is in the instance folder.
- requirement.txt lists your dependencies

Roles and Functionalities

Admin

The **Admin** is the superuser with full control. An admin account is created automatically upon database initialization.

- **Login:** Dedicated admin login.
- **Parking Lot Management:** Create new parking lots with details (location, price, total spots, address, pincode). Edit existing lot details. Delete lots (only if all spots are empty).
- **Spot Management:** Automatically generates Spots based on total_spots in a Lot. View status of individual parking spots.
- **User Management:** View a list of all registered users and their details, including spots used.

- **Dashboard:** Comprehensive view of parking lot and spot status, summary charts, and all parking records.

User

Users need to **register** and **login** to use the application.

- **Registration/Login:** Standard registration process with email, password, full name, address, and pincode.
- **Dashboard:** View available parking lots. Access personal parking history.
- **Reserve Parking:** Select an available parking lot; the system auto-allocates the first available spot. Users cannot manually pick a spot.
- **Occupy Spot:** Mark a reserved spot as "occupied" once parked (`start_time` recorded).
- **Release Spot:** Mark an occupied spot as "released" once vacated (`end_time` recorded).
- **Parking History:** View a detailed history of all past parking sessions, including duration and cost.

Video Link:

<https://drive.google.com/file/d/1zD6KAjGzEn9kZC-ftGZvXBZzcMevNNKn/view?usp=sharing>